AI-Powered Unified Sri Lankan Tourism Platform Using GraphRAG and LLM Orchestration

COMMON INTEGRATED SOLUTION REPORT

R26-IT-050

Team member	Student ID	Individual component
Deniyawatte D.G.A.D.N	IT22895196	C1 Intelligent Tour Package Generation
Kumarasinghe K.G.A.M.S.D.C.	IT22224866	C2 Context-Aware Itinerary Optimizer
Wijesinghe I.D.W.	IT22555458	C3 Intelligent Mobility, Route-Risk & Vehicle Selection
Madushan M.	IT22214720	C4 AI Landmark Recognition & Tourist Information

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specializing in Information Technology

Sri Lanka Institute of Information Technology

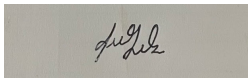
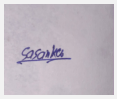
Sri Lanka

11 October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Deniyawatte D.G.A.D.N	IT22895196		11 October 2026
Kumarasinghe K.G.A.M.S.D.C.	IT22224866		11 October 2026
Wijesinghe I.D.W.	IT22555458		11 October 2026
Madushan M.	IT22214720		11 October 2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Dilani Lunugalage		

Role and name	Signature	Date
Co-supervisor: Ms. Dulani Maheshika Jayasinghe		

ABSTRACT

This project develops a unified Sri Lankan tourism decision-support prototype that connects hotel and activity matching, personalised itinerary planning, route and vehicle advice, and landmark recognition with a contextual assistant. The common problem is the effort required to turn separate travel preferences and information sources into a consistent journey. The solution combines a shared application, graph retrieval, predictive models, deterministic constraints, and optional language services. Express and Flask interfaces connect the components while local datasets, Neo4j, and external providers supply supporting information. Recorded component evaluations show route-severity test accuracy of 88.20 percent, itinerary v1 relevance accuracy of 68.33 percent on 60 human-reviewed pairs, and landmark SVM validation accuracy of 93.18 percent. These results measure different tasks and do not constitute overall platform accuracy. A separate itinerary extension candidate fails its activation criteria. The implementation provides explicit responses for missing matches, infeasible plans, and unavailable route comparisons. Remaining limitations include uneven data provenance and coverage, landmark training-to-serving differences, and the absence of a complete group-level user evaluation. The principal contribution is a common structure for connecting travel decisions while keeping their constraints and uncertainty explicit. Further work should prioritise reproducible deployment, consistent trip handoffs, independent component evaluation, and recorded end-to-end validation before wider deployment.

Keywords: Sri Lankan tourism, GraphRAG, itinerary planning, route advice, landmark recognition

ACKNOWLEDGEMENTS

We thank our supervisor, Ms. Dilani Lunugalage, and co-supervisor, Ms. Dulani Maheshika Jayasinghe, for their guidance. We acknowledge the academic staff of the Sri Lanka Institute of Information Technology and our project teammates for their support in developing the components and bringing them into a common application. We are grateful to our families for their encouragement. The sources and software evidence used in this report are acknowledged in the references.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1
1.1 Background and context 1
1.2 Stakeholders and user needs 1
1.3 Alignment with the Sustainable Development Goals 2
1.4 Purpose and scope of the common report 2
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 3
2.1 Overall problem definition 3
2.2 Root causes and overall research gap 3
2.3 Subproblem and component mapping 3
2.4 Overall aim and group objectives 4
2.5 Scope assumptions and constraints 4
3 OVERALL SOLUTION STRUCTURE 5
3.1 Integrated solution overview 5
3.2 High level architecture 5
3.3 Component 1 summary 6
3.4 Component 2 summary 7
3.5 Component 3 summary 7
3.6 Component 4 summary 7
3.7 End to end user and data flow 8
4 COMPONENT INTEGRATION AND COHESIVE OPERATION 9
4.1 Integration strategy 9
4.2 Interfaces data contracts and shared models 9
4.3 Shared nonfunctional requirements 9
4.4 Integrated workflow and exception handling 10
4.5 Team contribution and dependency matrix 11

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 12
5.1 Common technology stack and infrastructure 12
5.2 Development and integration process 12
5.3 Deployment and operational configuration 12
5.4 Evidence of team execution 13
6 INTEGRATED TESTING EVALUATION AND VALIDATION 14
6.1 Evaluation framework and success criteria 14
6.2 Integration and end to end testing 14
6.3 Overall results 15
6.4 Stakeholder and user validation 16
6.5 Achievement of common objectives 16
6.6 Integrated limitations and threats 17
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 18
7.1 Value proposition and beneficiaries 18
7.2 Market alternatives and differentiation 18
7.3 Business model and feasibility 18
7.4 Adoption intellectual property and compliance 19
7.5 Sustainable Development Goal contribution 19
8 CONCLUSION AND FUTURE DIRECTIONS 20
8.1 Overall conclusion 20
8.2 Principal integrated contribution 20
8.3 Future directions 20
REFERENCES 21
APPENDICES 22
Appendix A Integrated interface specification 22
Appendix B Team contribution declaration 23

LIST OF FIGURES

Figure 3.1. Shared application and component service boundaries. 6

LIST OF TABLES

Table 2.1. Subproblems and component responsibilities. 4

Table 4.1. Component dependencies and shared responsibilities. 11

Table 6.1. Integrated verification coverage. 15

Table 6.2. Recorded component results and interpretation. 15

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
CSV	Comma-separated values
GraphRAG	Graph retrieval augmented generation
JSON	JavaScript Object Notation
LLM	Large language model
POI	Point of interest
SDG	Sustainable Development Goal
SVM	Support-vector machine
TFLite	TensorFlow Lite

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Planning a visit to Sri Lanka involves several related decisions. A traveller needs to choose accommodation and activities, organise available time, select transport, and understand the places visited. These decisions influence each other. A suitable hotel can still be inconvenient for the selected activities, and an attractive list of destinations may exceed the time available. Our project addresses this coordination problem through a shared tourism application rather than treating each decision as a separate task.

The R26-IT-050 repository contains four functional areas: graph-based hotel and package matching, personalised itinerary optimisation, route and vehicle advice, and image-based landmark recognition with a contextual assistant [1]. Artificial intelligence (AI) supports selected decisions, while validation rules and structured data constrain the outputs. The project title describes the intended direction of graph retrieval augmented generation (GraphRAG) and large language model (LLM) orchestration. In the implementation, graph retrieval, predictive models, deterministic rules, and generated language have different responsibilities; not every component performs GraphRAG.

1.2 Stakeholders and user needs

The primary user is a traveller who wants practical help before and during a Sri Lankan journey. Before departure, that user may know a district, spending limit, preferred activities, and group size but may not know which combination is feasible. During a visit, the same user may want to identify a landmark and ask about its history or visitor information.

Secondary stakeholders include accommodation providers, tourism operators, local communities, and administrators who maintain information. The application requirements imply a need for understandable recommendations, visible constraints, and recoverable errors. These are design needs, not findings from a completed representative interview study. A group-level user evaluation is still needed to

establish which tasks matter most, how much time the application saves, and whether its explanations are understood.

1.3 Alignment with the Sustainable Development Goals

The intended social contribution relates to Sustainable Development Goal (SDG) 8, particularly target 8.9 on tourism that supports employment, local culture, and products [2]. Helping visitors discover appropriate local experiences could support that direction, but recommendations alone do not establish an economic benefit.

SDG 12, including target 12.b on monitoring sustainable tourism, also provides a useful frame [3]. The system would need measures such as destination distribution, provider visibility, and the effects of route choices before making sustainability claims. No measured reduction in emissions or increase in local income is claimed in this research.

1.4 Purpose and scope of the common report

This common report explains the overall problem, division into four components, shared architecture, interfaces, integration behaviour, and available evaluation evidence. It complements the individual theses, which should provide detailed component methods and experiments. The central question is whether these capabilities can form a coherent decision-support workflow while preserving the meaning and limitations of their outputs.

The report distinguishes implemented behaviour from proposed improvements. Recorded model results are used within their original evaluation scope. They are not combined into a single platform accuracy, because recognition, relevance prediction, and risk classification solve different tasks.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

The common problem is the effort required to turn travel preferences and fragmented information into a consistent journey plan. Travellers must reconcile accommodation, activities, travel time, transport suitability, and destination information. Our research focuses on decision continuity: one stage should support the next without silently changing locations, dropping constraints, or overstating confidence.

2.2 Root causes and overall research gap

Inconsistent place representations, separate data sources, incomplete coverage, and unstructured preferences complicate coordination. Image recognition adds uncertainty when the system receives a scene outside its supported classes. The project addresses an integration gap by connecting retrieval, planning, transport, and interpretation through explicit interfaces. The repository supports this implementation contribution [1], but does not establish that no comparable platform exists.

2.3 Subproblem and component mapping

Table 2.1 maps the four subproblems to separate responsibilities. Hotel/package matching answers what accommodation and activities fit the request. Itinerary optimisation addresses which places fit the available time and in what order. Route and vehicle advice considers transport requirements and available historical road context. Landmark recognition and conversation support interpretation during the visit.

Table 2.1. Subproblems and component responsibilities.

Component	Primary input	Main output
C1 Hotel and package matching	Natural-language preferences and optional stay details	Graph-matched hotels and activities
C2 Smart itinerary planning	Interests, candidate places and time constraints	Feasible ordered visit plan
C3 Route and vehicle advisor	Endpoints, budget, passengers and category	Analysed route and suitable vehicles
C4 Landmark and assistant	Landmark photograph and questions	Predicted identity and contextual information

The components are interdependent at the journey level, although a single shared data transaction across all four is not yet demonstrated. That distinction matters when evaluating how complete the integration is.

2.4 Overall aim and group objectives

The common aim is to develop and evaluate a unified Sri Lankan tourism decision-support prototype that combines structured retrieval, predictive methods, and contextual language assistance.

Objective O1 is to expose the four capabilities through a common application and explicit service interfaces. O2 is to preserve traveller constraints through package, itinerary, and transport decisions. O3 is to support recommendations with available structured evidence and communicate uncertainty. O4 is to assess component performance alongside interface and system behaviour. O5 is to identify the operational and data requirements for responsible deployment. Section 6.5 maps these objectives to the current evidence.

2.5 Scope assumptions and constraints

This research prototype depends on configured services, compatible model artefacts, and usable place identifiers. It is not a booking or emergency-response service. Coverage differs by component: itinerary evidence concerns bounded catalogues, landmark recognition covers 17 classes, and road evidence is incomplete [4]–[6]. A confidence score cannot guarantee recognition or road safety.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

The application supports a journey from preference entry to practical planning and on-site information. Hotel matching retrieves related accommodation and activities. Itinerary planning then addresses visit relevance and time feasibility. Route and vehicle analysis adds travel context, while the landmark assistant supports questions during a visit.

The unifying idea is to preserve the traveller's intent while changing the representation used to solve each task. Free text becomes structured filters, places become an ordered schedule, endpoints become a driving route, and a photograph becomes a landmark label. Each transformation needs its own validation because an error can affect later decisions.

3.2 High level architecture

Figure 3.1 shows the service boundaries and dependencies observed in the repository [1], [7], [8]. The React Native/Expo frontend communicates with an Express application and a Python Flask service. Neo4j supports graph queries, while local CSV files and saved model artefacts support the analytical components. External routing, geocoding, weather, pricing, and language services provide additional functions when configured.

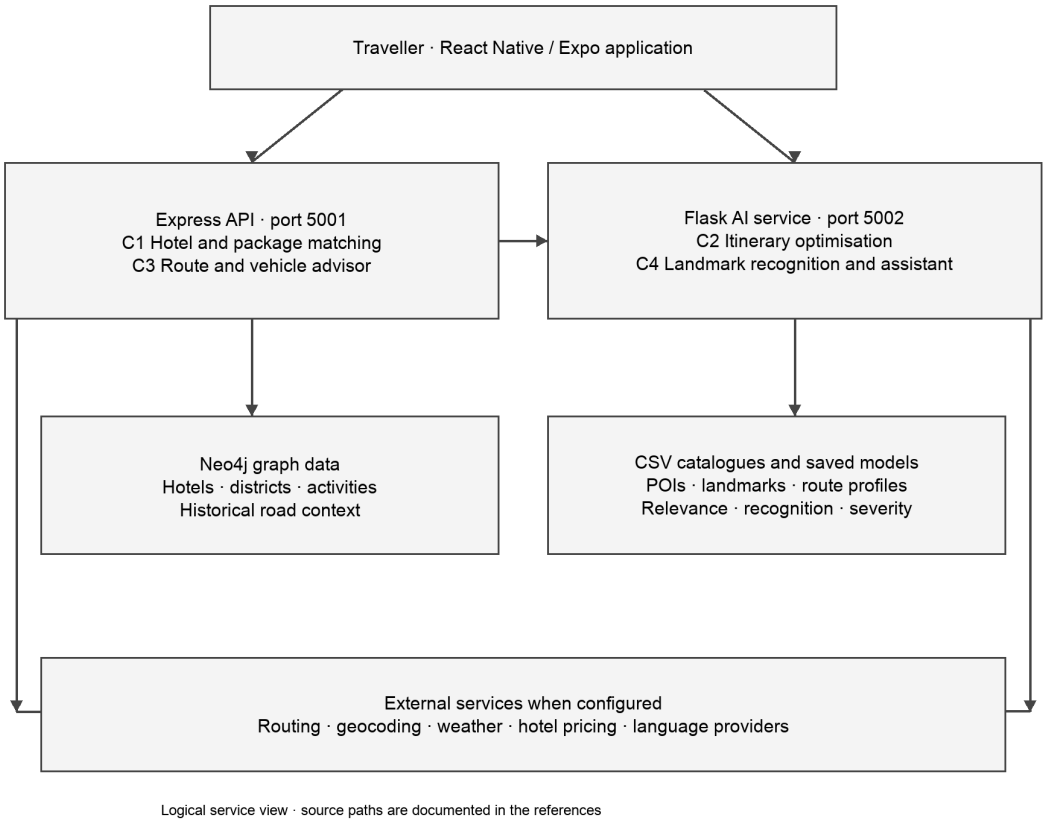


Figure 3.1. Shared application and component service boundaries.

The diagram represents logical connections, not a claim of a production deployment. Hotel and safety graphs serve different domains. The landmark assistant uses CSV metadata as context; it does not traverse the hotel graph. The application therefore combines several retrieval and inference methods rather than relying on one universal knowledge graph.

3.3 Component 1 summary

Component C1 handles hotel and package recommendations. A prompt is converted into preferences, and Neo4j queries match hotels, districts, and nearby activities. The query supports filters such as hotel category, grade, food type, activity category, price level, and visitor suitability. It returns bounded sets of hotels and activities [7].

The current controller ranks packages by matching activity count and then room count, selects up to three, and constructs a day-wise plan deterministically. Although a separate LLM itinerary-text helper exists, its presence does not establish that it is active

in this controller. The component also exposes hotel pricing for selected stay details. Its value to the common system is a structured starting point for travel planning.

3.4 Component 2 summary

Component C2 provides personalised itinerary planning. The Python service uses a bounded place catalogue, relevance filtering, ranking, and time-feasibility logic. The evaluation materials describe a relevance classifier based on user interests and verified point-of-interest (POI) tags. Candidate ordering and scheduling remain separate from the classifier [4].

The service preserves locked stops and returns explicit infeasibility when constraints cannot be satisfied. Its output supports a visit plan rather than simply a list of popular places. A later v2 relevance candidate has separate evaluation records and fails its stated activation gate; that candidate must not be described as a validated improvement [5].

3.5 Component 3 summary

Component C3 is the route and vehicle advisor associated with WIJESINGHE I.D.W., IT22555458. It accepts start and destination information, budget, passenger count, and vehicle preferences. Routing, weather enrichment, historical graph evidence, and a Gradient Boosting classifier contribute different forms of context [6].

The classifier predicts an internal historical hazard-severity class. Vehicle candidates are assessed against constraints and available road attributes. Alternative routes are compared only when distinct usable evidence is available. Otherwise, the normal route can still be analysed and mapped while the response states that lower-risk comparison is unavailable. This prevents missing comparison evidence from being treated as proof that a route is safest.

3.6 Component 4 summary

Component C4 recognises supported landmarks and provides a contextual AI assistant. The notebook compares a convolutional neural network, MobileNetV2, a Vision Transformer, and a support-vector machine (SVM) using deep features [8]. The application receives an image, predicts a label, and connects it to landmark metadata.

The assistant supplies that metadata to a configured language provider and has local responses for common topics when the provider is unavailable. Recognition and conversation are separate evaluation problems. A correct label does not guarantee a factual generated answer. Training-to-serving differences also require attention before notebook accuracy can be treated as deployed recognition performance.

3.7 End to end user and data flow

A representative workflow begins when a traveller requests a package for a preferred district and activity type. The system extracts filters, retrieves matching hotels and activities, and returns choices. The traveller can then use itinerary planning to organise visits within time constraints and request transport advice for the relevant journey. At a destination, a photograph can open contextual landmark information and follow-up conversation.

This describes the intended continuous user journey. Existing endpoints support its component operations, including itinerary safety interfaces, but a recorded live session carrying every selection automatically through all four modules is not available. A shared trip identifier and explicit handoff contract are therefore priorities for stronger integration, rather than assumptions about existing behaviour.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

The implementation separates presentation, orchestration, data access, and model inference. This allows each component to use an appropriate method while exposing a service boundary. Express coordinates the JavaScript services; Flask handles itinerary optimisation and landmark requests. Neo4j access and external provider calls are encapsulated in service modules [1], [7], [8].

Integration should proceed from stable request and response contracts to combined workflows. The current repository contains service and integration tests around itinerary and safety operations. These provide a basis for regression control, but they do not replace a full release test with all providers and models enabled.

4.2 Interfaces data contracts and shared models

Requests use JavaScript Object Notation (JSON) for structured planning operations and multipart image upload for recognition. The Express routes include POST /api/generate-package, POST /api/hotel-price, POST /api/itinerary/optimize, POST /api/safety/recommend-route, and POST /api/safety/recommend-itinerary. Flask exposes POST /api/landmark/predict and POST /api/landmark/chat [1], [7], [8]. Appendix A records their integration responsibilities.

Shared meaning is more important than matching field names alone. A trip budget, a hotel stay total, and a vehicle estimate cover different costs. Likewise, route duration and total itinerary duration are not interchangeable. The consuming interface should show units, currency, scope, and missing values explicitly. Identifiers should remain stable when a user revises a plan, and client-provided road risk should not override backend-derived context.

4.3 Shared nonfunctional requirements

Reliability requires bounded provider calls, useful failure messages, and preservation of valid partial results. Performance should be measured for complete user tasks, because an inexpensive classifier may still sit behind slow routing or language

requests. No platform-wide response-time service level is established by the available evidence.

Privacy concerns include uploaded images, conversation text, location input, and travel preferences. Operational design should minimise retention, avoid secrets in client code, and explain external processing. Security testing should cover request validation, image size and decoding, administrative access, and error disclosure. Accessibility requires readable labels, non-colour status cues, keyboard access on web, and testing with assistive technology. These are shared acceptance requirements; a formal accessibility or security certification is not claimed.

4.4 Integrated workflow and exception handling

Failure should be handled where its meaning is understood. If graph matching returns no packages, the package controller returns a clear no-match response. If required itinerary stops already exceed the time allowance, the planner should expose infeasibility rather than silently omit them. If distinct route evidence is absent, the transport workflow returns analysis without inventing an alternative comparison.

Landmark recognition validates the uploaded image and can reject low-confidence predictions. The assistant can use local topic responses when a language provider fails. However, each fallback has a narrower capability than the normal path. The frontend should distinguish an unavailable service, an empty result, and an uncertain prediction so the traveller knows whether to retry, revise input, or seek another source.

4.5 Team contribution and dependency matrix

Table 4.1 assigns responsibilities by component. It is a functional dependency matrix, not a substitute for a signed record of individual work. The team contribution declaration appears in Appendix B.

Table 4.1. Component dependencies and shared responsibilities.

Area	Primary responsibility	Shared dependency
C1	Preference extraction, hotel and activity retrieval	Place identity, dates and pricing scope
C2	Relevance and time-feasible visit ordering	POI catalogue, travel times and locked stops
C3	Route evidence and vehicle suitability	Resolved locations, road context and budgets
C4	Image recognition and landmark dialogue	Label metadata and provider availability
Team	Interface consistency and release verification	Common frontend, configuration and evaluation

Research ownership includes data preparation, method selection, evaluation, and interpretation. Integration ownership includes documenting contracts and checking that another component can consume the result correctly. Both are needed for a credible common solution.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

The shared stack combines React Native/Expo, Node.js/Express, Python/Flask, Neo4j, CSV datasets, and saved machine-learning artefacts [1]. The JavaScript layer is suited to the frontend-facing application programming interfaces (APIs) and provider orchestration. Python supports the model and optimisation ecosystem. Neo4j represents relationships used for hotel/activity retrieval and historical safety evidence.

External dependencies include routing, geocoding, weather, hotel-pricing, and optional language services. MongoDB is used where wider backend functionality requires it. This mixed stack supports a working research environment, but increases configuration and compatibility requirements. A deployment must preserve model versions, dependency versions, and dataset provenance together.

5.2 Development and integration process

The repository organises routes, controllers, services, AI modules, frontend screens, datasets, and tests as separate concerns. This structure makes interface changes easier to trace than a single combined script. Version control can preserve changes to source and reproducible evaluation artefacts, while environment-specific credentials remain outside the repository.

For release work, a useful sequence is to freeze contracts, test individual services, exercise combined workflows with controlled dependencies, and then repeat critical scenarios against configured live services. A successful local test should be recorded with the commit, environment, dataset version, and provider mode. The available files support the implementation structure, but do not establish a complete history of sprint reviews or continuous-integration execution.

5.3 Deployment and operational configuration

The documented local setup uses the Express API on port 5001 and the Python AI service on port 5002 [1]. Backend dependencies are installed through the package manifest, while the Python service requires a compatible environment and its pinned

dependencies. Neo4j must be configured and the relevant graph data loaded for graph-dependent workflows.

Operational configuration includes database connection settings, provider credentials, model paths, and optional language-service settings. Before a demonstration, the team should verify health endpoints and complete one small request per component. Backups should include source, dataset manifests, graph import inputs, and model artefacts, with credentials stored separately. Restore and load testing remain deployment tasks rather than completed research outcomes.

5.4 Evidence of team execution

Concrete execution evidence includes the service modules, frontend screens, notebook outputs, evaluation reports, and automated test files stored in R26-IT-050. Together, they show implementation across all four problem areas and identify the main integration boundaries [1], [4]–[9].

These artefacts do not by themselves establish individual hours, meeting attendance, or equal contribution. No meeting minutes, task-board export, or signed contribution log is asserted here. The team should confirm the ownership matrix against its records before signing the declaration. Appendix B separates functional responsibilities from this required personal confirmation.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

Evaluation has three levels. Component evaluation measures the specific learning or retrieval task. Interface evaluation checks request validation, output structure, and failure handling. End-to-end evaluation checks whether a traveller can complete a coherent task across the application. Evidence at one level cannot be substituted for another.

Proposed group acceptance criteria include preserving all valid locked itinerary stops, exposing infeasible requests, showing unavailable route comparisons, maintaining the selected landmark in conversation, and preserving cost units and scope between screens. A complete study should also measure task completion, total response time, correction rate, and explanation comprehension. These system-level criteria require a recorded acceptance run and user study; they are not presented as already achieved thresholds.

6.2 Integration and end to end testing

The repository includes automated tests for location similarity, safety input validation, hotel pricing, itinerary services, and itinerary-safety integration [9]. These files provide repeatable checks around selected contracts. Their existence is evidence of test implementation, not a statement that all tests passed in the current environment. No new full-system test execution is reported here.

Table 6.1 summarises the coverage needed for a group release. Live-provider tests should be separated from deterministic tests so an external outage is not confused with a logic regression.

Table 6.1. Integrated verification coverage.

Scenario	Available evidence	Remaining verification
Package and pricing	Controller and pricing test files	Complete live search and stay-price flow
Itinerary and safety	Service and integration tests	UI handoff with consistent stops and dates
Route and vehicle	Validation and route tests	Provider-backed alternatives and no-match cases
Landmark and assistant	Notebook and endpoint implementation	Serving parity and factual dialogue study
Complete journey	Common application and service boundaries	Recorded four-component acceptance session

6.3 Overall results

Table 6.2 brings the recorded results together without averaging them. The route classifier reports 88.20 percent final-test accuracy and macro-F1 0.8780 on its internal dataset [6]. The v1 itinerary relevance evaluation reports 68.33 percent accuracy and macro-F1 0.5196 on 60 human-reviewed profile–POI judgements [4]. A separate v2 extension evaluation reports 60.00 percent accuracy and macro-F1 0.5775, failing its activation gate [5]. The landmark notebook records SVM validation accuracy of 93.18 percent and weighted-F1 93.23 percent [8].

Table 6.2. Recorded component results and interpretation.

Evaluation	Recorded result	Scope
Route severity	Accuracy 88.20%; macro-F1 0.8780	Internal proxy-labelled test set
Itinerary v1 relevance	Accuracy 68.33%; macro-F1 0.5196	60 reviewed pairs; three held-out profiles
Itinerary v2 extension	Accuracy 60.00%; macro-F1 0.5775	60 single-reviewer pairs; gate failed
Landmark SVM	Accuracy 93.18%; weighted-F1 93.23%	Reused validation split; 17 classes
Hotel/package matching	Structured retrieval implemented	No common quantitative benchmark

These figures support bounded component findings. They do not show 93.18 percent accuracy for the integrated platform. The main group result is an implemented

collection of connected decision-support services with explicit constraints and uneven evaluation maturity.

6.4 Stakeholder and user validation

Human-reviewed relevance labels provide a limited form of validation for itinerary recommendations. The v1 record reports reviewer agreement of 53 out of 60 pairs before adjudication, while the v2 extension uses one reviewer [4], [5]. Neither is a usability study of the full application.

A group-level study should recruit travellers with varied familiarity with Sri Lanka and ask them to complete a package-to-visit task. Observation should capture incorrect handoffs, repeated data entry, misunderstood warnings, and recovery from failed services. Post-task questions should distinguish satisfaction from objective task success. No participant count, satisfaction percentage, or interview quotation is invented for this report.

6.5 Achievement of common objectives

O1 is partly achieved: the four capabilities and their service boundaries are implemented, but a complete recorded journey across them is still needed. O2 is partly achieved through component-specific constraints, with cross-component consistency requiring broader acceptance tests. O3 is partly achieved through structured retrieval, contextual explanations, and explicit missing-evidence behaviour; provenance and serving issues remain.

O4 is partly achieved because comparative component results and test suites exist, while overall usability and live integration evidence are incomplete. O5 is achieved as an analysis of deployment needs in this report, not as proof of commercial readiness. This assessment reflects the strength of the evidence rather than assigning completion from feature presence alone.

6.6 Integrated limitations and threats

The strongest system-level threat is inconsistent evidence quality. The safety training dataset has 598 records, with 463 lacking matching numeric road-profile data. Its original source provenance is not independently documented, and route-family aggregation is not exact traversed-segment matching [6]. The itinerary evaluations cover small profile sets and bounded catalogues [4], [5].

The landmark experiment reuses a validation partition during development. In addition, the serving code reconstructs an ImageNet feature extractor while the notebook SVM uses a fine-tuned backbone. Potential preprocessing differences in the TFLite path also require verification [8]. These issues can separate recorded experimental results from deployed behaviour.

External-service availability, stale visitor information, identity matching across datasets, and language-model errors affect the entire journey. An apparently successful response can still contain inconsistent dates, costs, or locations. A release should therefore prioritise contract and evidence checks before extending feature breadth.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The proposed value is a reduction in the effort needed to connect travel decisions. A traveller could move from an initial request to accommodation choices, a feasible visit plan, transport advice, and contextual destination information in one application. Local providers could benefit from clearer discovery when their information is represented accurately.

These benefits remain hypotheses until evaluated with users and providers. A credible value study should compare completion time, corrections, and confidence against the same task completed using separate tools. Provider visibility should also be monitored so the system does not repeatedly favour already well-represented destinations.

7.2 Market alternatives and differentiation

Relevant substitute approaches include separate booking tools, map applications, tourism information pages, and general conversational assistants. This is a functional comparison rather than a current market ranking. Our intended differentiation is continuity between structured Sri Lankan tourism data, planning constraints, transport context, and landmark conversation.

The approach is defensible only if the integration reduces user effort and improves consistency. Having four modules in one menu is insufficient. The main research and product question is whether information moves reliably between them and whether users can understand why a recommendation was made.

7.3 Business model and feasibility

A possible delivery model is a free traveller-facing planning service with optional provider partnerships or institutional support. Paid placement, if introduced, should remain clearly separate from recommendation suitability. No revenue, conversion rate, market size, or willingness-to-pay figure has been established.

Likely cost categories include hosting, external API requests, database operation, model maintenance, content verification, and support. The first feasibility milestone

should be a bounded pilot with measured usage and cost per completed journey. This would allow the team to evaluate affordability before expanding geographic coverage or promising continuous service.

7.4 Adoption intellectual property and compliance

Adoption depends on trustworthy local information, clear uncertainty, reasonable response time, and support for common devices and language needs. The team must establish rights to source images, datasets, software dependencies, and provider content before public distribution. A repository licence alone does not establish ownership of every embedded dataset.

A deployment review should define image and conversation retention, consent for external processing, access control, deletion procedures, and incident handling. Recommendations should remain advisory, and commercial pricing should identify what is included and when it was obtained. No legal compliance certification is claimed. Data provenance and user-facing explanations are practical prerequisites for responsible adoption.

7.5 Sustainable Development Goal contribution

The platform could support the tourism aims of SDG 8 through culturally relevant discovery and fairer visibility of local experiences [2]. That contribution should be assessed with indicators such as exposure across providers and destinations, rather than inferred from the number of recommendations produced.

For SDG 12, useful future measures include destination concentration and the resource implications of suggested journeys [3]. More recommendations can also increase crowding or encourage unnecessary travel. The system therefore needs monitoring and community input before its sustainability effect can be described as positive. These are proposed assessment measures, not observed impacts.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

The project connects accommodation and activity retrieval, itinerary planning, route and vehicle advice, and landmark conversation to address tourism planning as a shared task. Implementation and component evaluations support a coherent research prototype. They do not yet establish nationwide reliability or commercial readiness. Data provenance, serving consistency, and end-to-end validation remain the main priorities.

8.2 Principal integrated contribution

The principal contribution is a common application structure that uses different methods for successive travel decisions. Graph retrieval supports hotel and activity matching; relevance and scheduling methods organise visits; historical evidence informs qualified transport advice; and image recognition supplies context for landmark questions. Preserving the meaning of each output is essential: risk classification, relevance, recognition confidence, and factual dialogue require separate evaluation.

8.3 Future directions

The immediate priority is to reconcile training and serving artefacts, preserve dataset manifests, and confirm stable place identifiers. The team should then record a complete journey across the four modules, including provider failure and infeasible-input cases. A shared trip context should carry confirmed locations, dates, selected items, and cost scope between components.

The next research priority is independent evaluation: broader reviewed relevance data, untouched landmark test images, better documented road evidence, and factuality testing for assistant responses. A traveller study should measure task success and explanation comprehension. Geographic expansion and commercial trials should follow those checks, with explicit monitoring of data freshness, provider cost, and local impact.

REFERENCES

[1] R26-IT-050 Project Team, “Unified tourism platform application and configuration,” R26-IT-050 project repository, backend/src/app.js, frontend/App.js, and backend/README.md, source code and documentation, 2026. Local working copy inspected Oct. 11, 2026.

[2] United Nations Department of Economic and Social Affairs, “Goal 8,” Sustainable Development Goals. Accessed: Oct. 11, 2026. [Online]. Available: https://sdgs.un.org/goals/goal8

[3] United Nations Department of Economic and Social Affairs, “Goal 12,” Sustainable Development Goals. Accessed: Oct. 11, 2026. [Online]. Available: https://sdgs.un.org/goals/goal12

[4] R26-IT-050 Project Team, “Itinerary User–POI Relevance Classifier v1,” R26-IT-050 project repository, backend/src/ai_service/data/evaluation/user_poi_relevance_model_report_v1.md, evaluation report, 2026.

[5] R26-IT-050 Project Team, “Frozen v2 relevance candidate — single-reviewer extension evaluation,” R26-IT-050 project repository, backend/src/ai_service/data/evaluation/user_poi_relevance_v2_single_reviewer_report_v1.md, 2026.

[6] R26-IT-050 Project Team, “Safety-aware tourism backend and model comparison,” R26-IT-050 project repository, backend/README.md, backend/src/routes/safetyRoutes.js, and backend/src/ai-engine/data/model_comparison_results.csv, 2026.

[7] R26-IT-050 Project Team, “Hotel package retrieval and orchestration,” R26-IT-050 project repository, backend/src/controllers/hotelData.controller.js and backend/src/services/package.service.js, source code, 2026.

[8] R26-IT-050 Project Team, “Landmark recognition experiments and AI endpoints,” R26-IT-050 project repository, data-scripts/Landmark_Recognition.ipynb, backend/src/ai_service/app.py, and backend/src/ai_service/landmark_routes.py, notebook and source code, 2026.

[9] R26-IT-050 Project Team, “Backend service and integration tests,” R26-IT-050 project repository, backend/tests/route-location-similarity.test.js, safety.validation.test.js, liteapi-hotel-pricing.test.js, itinerary.service.test.js, and itinerary-safety.integration.test.js, test source files, 2026.

APPENDICES

Appendix A Integrated interface specification

The following contract summary identifies the integration boundary and the responsibilities of the caller and receiving service. Exact field definitions remain in the corresponding source files [1], [7], [8].

Interface	Request purpose	Response or constraint
/api/generate-package	Prompt and optional stay details	Matching packages or explicit no-match
/api/hotel-price	Selected hotel and validated stay	Stay pricing with currency context
/api/itinerary/optimize	Preferences and planning constraints	Optimised plan or infeasibility
/api/safety/recommend-route	Locations and transport preferences	Route analysis and optional vehicles
/api/safety/recommend-itinerary	Itinerary travel context	Safety analysis through shared services
/api/landmark/predict	Multipart image	Landmark prediction or rejection
/api/landmark/chat	Landmark context and question	Provider or local contextual response

The first five paths belong to the Express API; the final two belong to Flask. All listed operations use POST. A release contract should additionally freeze required fields, units, identifier formats, error codes, provider-mode indicators, and sample responses. Cross-component tests should check that the frontend preserves those meanings when moving between screens.

Appendix B Team contribution declaration

The component responsibilities in Table 4.1 provide the basis for the team declaration. Deniyawatte D.G.A.D.N (IT22895196) is responsible for Intelligent Tour Package Generation; Kumarasinghe K.G.A.M.S.D.C. (IT22224866) for Context-Aware Itinerary Optimizer; Wijesinghe I.D.W. (IT22555458) for Intelligent Mobility, Route-Risk & Vehicle Selection; and Madushan M. (IT22214720) for AI Landmark Recognition & Tourist Information. Functional ownership does not imply that all shared integration work was performed by one member.

Each member should confirm their research tasks, dataset work, implementation, evaluation, and contribution to the common interfaces. Shared contributions include coordinating data formats, maintaining application navigation, reviewing integration behaviour, and preparing the common report. Supporting evidence should be linked to repository changes or agreed project records.

Generative AI assistance through Codex was used to inspect repository artefacts, organise and draft report content, format the supplied Word template, create the architecture diagram, and check document layout. No new model training, participant study, or full-system execution is claimed as part of report preparation. The team remains responsible for verifying the report and disclosing any additional AI use under the applicable institutional policy.

Member confirmation and signing dates remain blank in the declaration. No signatures have been generated or inserted.